

《数据库系统原理》报告

数据库系统原理作业 7

张鲈沅

北京交通大学计算机科学与技术学院 AI2201 班 22281052@bjtu.edu.cn

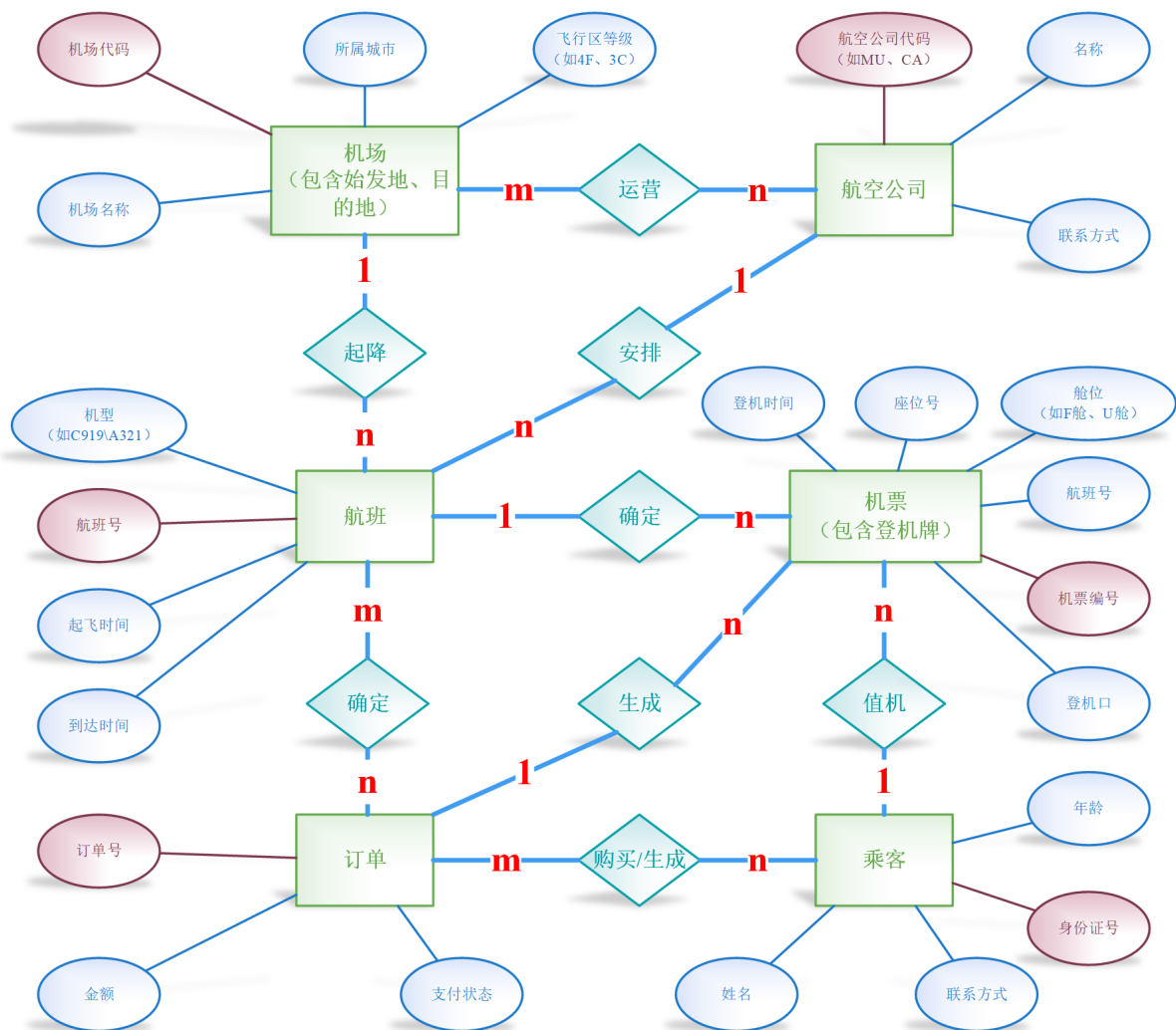


图 1: 航空公司航班查询业务模型 E-R 图，来自于作业 6

1 题目 1: 存储过程和触发器实验

题目要求:

1. 请在你选用的数据库平台上，针对你的应用场景，对如下操作至少各实现一个存储过程：

- 单表或多表查询
- 数据插入
- 数据删除
- 数据修改

2. 通过 ODBC、OLEDB、JDBC 或任意其他的途径，在前端程序（C/S 或 B/S 模式）中调用所实现的后台存储过程。

3. 在你的案例场景中，分别设计并实现一个由数据插入、数据更新、数据删除所引发的触发器（前触发或后触发都可以），测试触发器执行效果。

本实验的核心在于设计、实现并验证数据库存储过程和触发器在航空公司航班查询系统中的应用。这一部分整体以 MySQL 实现，前端交互通过 Java 语言和 JDBC 驱动实现。

1.1 存储过程的设计

存储过程作为数据库服务器端预编译的 SQL 代码单元，能够封装复杂的业务逻辑，提高执行效率并减少网络通信。根据作业要求，我针对数据库的四种基本操作（查询、插入、修改、删除）设计了相应的存储过程。

查询存储过程 (GetFlightsByRoute):

该存储过程旨在根据用户提供的出发机场代码和到达机场代码，检索相关的航班信息。为了提供更全面的信息，该过程执行了一个多表连接查询，涉及 Flight 表（存储航班基本信息）、Airport 表（通过出发地和目的地代码关联，获取机场名称和城市）以及 Airline 表（通过航空公司代码关联，获取航空公司名称）。其定义接受两个输入参数：IN in_departure_airport_code VARCHAR(10) 和 IN in_arrival_airport_code VARCHAR(10)。查询结果包含航班号、机型、起降时间、出发机场名称、出发城市、到达机场名称、到达城市以及航空公司名称。

```
1 DROP PROCEDURE IF EXISTS GetFlightsByRoute;
2 CREATE PROCEDURE GetFlightsByRoute(
3     IN dep_airport VARCHAR(10),
4     IN arr_airport VARCHAR(10)
5 )
6 BEGIN
7     SELECT
8         f.flight_number,
9         f.air_model,
10        f.departure_time,
```

```

11         f.arrival_time,
12         a1.airport_name AS departure_airport_name,
13         a1.city AS departure_city,
14         a2.airport_name AS arrival_airport_name,
15         a2.city AS arrival_city,
16         a1.airline_name
17     FROM Flight f
18     JOIN Airport a1 ON f.departure_airport = a1.airport_code
19     JOIN Airport a2 ON f.arrival_airport = a2.airport_code
20     JOIN Airline al ON f.airline_code = al.airline_code
21     WHERE f.departure_airport = dep_airport
22           AND f.arrival_airport = arr_airport;
23 END //
```

数据插入存储过程 (AddPassenger):

此存储过程用于向 Passenger 表中添加新的乘客记录。它接受四个输入参数：IN in_id_card VARCHAR(20) (乘客身份证号, 主键)、IN in_passname VARCHAR(50) (乘客姓名)、IN in_age INT (乘客年龄) 和 IN in_phone VARCHAR(100) (乘客电话)。存储过程体内部执行一条标准的 INSERT INTO Passenger 语句。数据库的 PRIMARY KEY 约束将自动处理 id_card 的唯一性, chk_age 约束将验证年龄的有效性。

```

1 DROP PROCEDURE IF EXISTS AddPassenger;
2 CREATE PROCEDURE AddPassenger(
3     IN in_id_card VARCHAR(20),
4     IN in_passname VARCHAR(50),
5     IN in_age INT,
6     IN in_phone VARCHAR(100)
7 )
8 BEGIN
9     INSERT INTO Passenger (id_card, passname, age, phone)
10    VALUES (in_id_card, in_passname, in_age, in_phone);
11 END //
```

数据修改存储过程 (UpdatePassengerPhone):

该存储过程的功能是更新指定乘客的电话号码。它接受两个输入参数：IN in_id_card VARCHAR(20) (用于定位要更新的乘客记录) 和 IN in_phone VARCHAR(100) (新的电话号码)。存储过程体内部执行一条 UPDATE Passenger SET phone = ... WHERE id_card = ... 语句。

```

1 DROP PROCEDURE IF EXISTS UpdatePassengerPhone;
2 CREATE PROCEDURE UpdatePassengerPhone(
3     IN in_id_card VARCHAR(20),
```

```

4      IN in_phone VARCHAR(100)
5  )
6  BEGIN
7      UPDATE Passenger
8      SET phone = in_phone
9      WHERE id_card = in_id_card;
10 END //

```

数据删除存储过程 (DeletePassenger):

此存储过程用于从 Passenger 表中删除指定的乘客记录。它接受一个输入参数: IN in_id_card VARCHAR(20)。存储过程体内部执行一条 DELETE FROM Passenger WHERE id_card = ... 语句。需要特别注意的是, 由于 Passenger 表中的 id_card 被 Ticket 表和 Orderinfo 表作为外键引用, 直接删除可能会因为违反外键约束而失败, 除非外键设置了 ON DELETE CASCADE 或相关联的子表记录已被预先移除。

```

1 DROP PROCEDURE IF EXISTS DeletePassenger;
2 CREATE PROCEDURE DeletePassenger(
3     IN in_id_card VARCHAR(20)
4 )
5 BEGIN
6     DELETE FROM Passenger
7     WHERE id_card = in_id_card;
8 END //

```

所有存储过程在 MySQL 中创建时, 均使用了 DELIMITER // 命令临时更改语句结束符, 以确保存储过程定义体内的分号被正确解析。创建完成后, 再使用 DELIMITER ; 恢复默认结束符。因此, 在 MySQL Line Client 里面输入时, 需要以下面方式输入。

```

1 DELIMITER //
2 \. D:/文件路径/exp7_2.sql
3 DELIMITER ;

```

1.2 前端程序通过 JDBC 调用存储过程

为了在客户端应用程序中与数据库服务器端的存储过程进行交互, 通过询问王梓旭同学, 我选择使用 Java 语言和 JDBC (Java Database Connectivity) API。实验中编写了一个名为 SPCaller.java 的 Java 类, 该类负责建立数据库连接并调用上述实现的四个存储过程。

项目依赖与配置:

在 IntelliJ IDEA 项目中, 我通过手动添加 JAR 文件的方式引入了 MySQL Connector/J 驱动 (版本 mysql-connector-j-9.0.0.jar) 及其依赖的 protobuf-java-4.26.1.jar。

这些库文件被添加到模块的类路径中 (如图2), 使得 Java 程序能够加载 MySQL 的 JDBC 驱动。

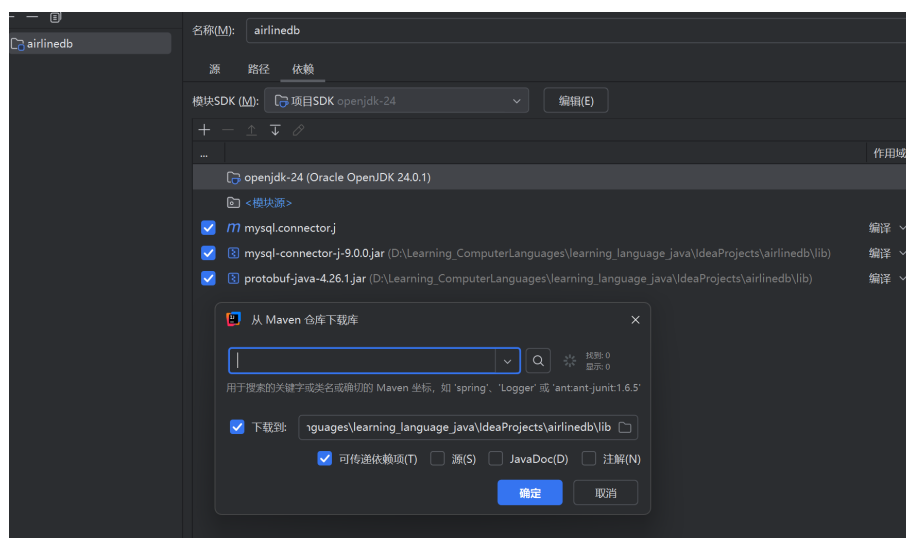


图 2: 项目依赖与配置

数据库连接建立:

SPCaller 类中包含一个私有静态方法 `getConnection()`, 该方法使用 `DriverManager.getConnection(DB_URL, USER, PASS)` 来建立与 `airlinedb` 数据库的连接。连接 URL、用户名和密码作为静态常量在类中定义, 实际使用时需替换为真实凭据。连接 URL 中添加了 `?serverTimezone=UTC` 参数以兼容较新的 JDBC 驱动版本。

存储过程调用机制:

对于每个存储过程, 实现了一个对应的公共静态 Java 方法 (`callGetFlightsByRoute`, `addPassenger` 等)。这些方法接收一个 `Connection` 对象和存储过程所需的输入参数。

- 使用 `Connection.prepareStatement` 方法创建 `java.sql.CallableStatement` 对象。JDBC 的转义语法 `{call ...}` 确保了调用的可移植性。
- 使用 `CallableStatement.setX(parameterIndex, value)` 方法为存储过程的输入参数绑定具体值。 `parameterIndex` 从 1 开始计数。

• 执行与结果处理:

- 对于返回结果集的查询存储过程, 使用 `CallableStatement.execute()` 方法执行。如果该方法返回 `true`, 则通过 `CallableStatement.getResultSet()` 获取 `java.sql.ResultSet` 对象, 然后遍历结果集, 使用 `ResultSet.getX()` 获取每一行每一列的数据并进行展示。我还使用了 `ResultSetMetaData` 来动态获取列名, 以便更通用地打印表头。
- 对于执行数据操作语言 (DML) 的存储过程 (如 `AddPassenger`, `UpdatePassengerPhone`, `DeletePassenger`), 使用 `CallableStatement.executeUpdate()` 方法执行。

该方法返回受 DML 操作影响的行数，我可以根据此返回值判断操作是否成功执行。

资源管理与异常处理：

所有数据库资源（Connection, CallableStatement, ResultSet）的获取和使用都放置在 try-with-resources 结构中，以确保它们在操作完成或发生异常时能够被自动关闭，防止资源泄漏。同时，使用 try-catch 块捕获可能发生 SQLException，并打印错误信息以便调试。SPCaller 类的 main 方法作为测试入口，依次调用上述封装好的存储过程调用方法，并打印相应的提示信息和结果到控制台。

前端 Java 代码的整体结构展示了客户端如何通过标准的 JDBC 接口与数据库服务器端的存储过程进行交互，实现业务逻辑的分离。此处代码可见附录A。

1.3 JDBC 调用存储过程结果

通过执行前端 Java 程序 SPCaller.java，我对实现的四个存储过程进行了调用测试。控制台输出完整记录了连接建立、各存储过程调用及其返回结果或错误信息。

数据库连接：程序启动后，首先尝试连接数据库，输出”数据库连接成功!”，表明 Java 应用程序与 MySQL 数据库之间的通信通道已成功建立（如图3）。这是后续所有数据库操作的基础。

```
程序开始...
尝试连接到数据库...
数据库连接成功!
```

图 3: 建立通信通道

GetFlightsByRoute (查询存储过程) 调用分析：

- 第一次调用参数为 ("PEK", "PVG")，旨在查询从北京到上海的航班。根据输出，此调用成功执行，并返回了预期的航班数据（如航班号 CA1234，航空公司 Air China 等），证明了查询逻辑的正确性和多表连接的有效性。（如图4）
- 第二次调用参数为 ("ABC", "XYZ")，旨在查询一个不存在的航线。输出显示”此查询没有返回结果集”，这也是符合预期的，表明数据库中没有匹配的数据。（如图4）

```
--- 调用 GetFlightsByRoute 存储过程 (查询) ---
调用 GetFlightsByRoute for: PEK to PVG
找到的航班:
flight_number      air_model      departure_time  arrival_time    departure_airport_name  departure_city  arrival_airp
-----
CA1234             Boeing737      2024-04-10 16:00:00.0  2024-04-10 18:30:00.0  Beijing Capital International AirportBeijing
调用 GetFlightsByRoute for: ABC to XYZ
找到的航班:
flight_number      air_model      departure_time  arrival_time    departure_airport_name  departure_city  arrival_airp
-----
```

图 4: 查询存储过程调用

AddPassenger (插入存储过程) 调用分析：

- 第一次尝试插入乘客 ID 为”110101200001019999”的记录，显示插入成功。输出“AddPassenger 调用成功。受影响行数: 1”。(如图5)
- 第二次尝试插入乘客 ID 为”110101200001019999”的记录。根据输出，此操作失败（如图5），并抛出错误”调用 AddPassenger 存储过程时出错: Duplicate entry '110101200001019999' for key 'passenger.PRIMARY'”（如图6）。这表明该乘客 ID 在执行此 Java 程序之前已经存在于 Passenger 表中，违反了主键唯一性约束，数据库正确地拒绝了重复插入。

```

--- 调用 AddPassenger 存储过程（插入） ---
正在为: 测试用户 (110101200001019999) 调用 AddPassenger 存储过程
AddPassenger 调用成功。受影响行数: 1

尝试使用重复ID添加乘客:
正在为: 重复用户 (110101200001019999) 调用 AddPassenger 存储过程

```

图 5: 插入存储过程调用

UpdatePassengerPhone (修改存储过程) 调用分析:

- 第一次调用，尝试更新 ID 为”110101200001019999”的乘客的电话号码。输出显示”UpdatePassengerPhone 调用成功。受影响行数: 1”，表明更新操作成功。这间接证明了该乘客 ID 在数据库中是存在的。(如图6)
- 第二次调用,尝试更新一个不存在的乘客 ID (“999...”)。输出显示”UpdatePassengerPhone 存储过程已执行，但没有行受到影响。...”，这符合预期，因为没有记录匹配更新条件。(如图6)

```

--- 调用 UpdatePassengerPhone 存储过程（修改） ---
正在为 ID: 110101200001019999 调用 UpdatePassengerPhone 存储过程, 新电话为: 13900000000
UpdatePassengerPhone 调用成功。受影响行数: 1
正在为 ID: 9999999999999999999 调用 UpdatePassengerPhone 存储过程, 新电话为: 13012345678
调用 AddPassenger 存储过程时出错: Duplicate entry '110101200001019999' for key 'passenger.PRIMARY'
UpdatePassengerPhone 存储过程已执行，但没有行受到影响。乘客 ID '9999999999999999999' 可能不存在或电话号码已是最新。

```

图 6: 修改存储过程调用

DeletePassenger (删除存储过程) 调用分析:

针对之前插入的新乘客 ID (“110101200001018888”),程序尝试调用 DeletePassenger 存储过程。输出显示”DeletePassenger 调用成功。受影响行数: 1”。这表明删除操作成功执行, Passenger 表中对应 ID 的记录已被移除。(如图7)

```
--- 调用 DeletePassenger 存储过程（删除） ---  
正在为 ID: 110101200001018888 调用 DeletePassenger 存储过程  
DeletePassenger 调用成功。受影响行数: 1  
  
程序结束。
```

图 7: 删除存储过程调用

此结果证实了 DeletePassenger 存储过程的功能正确性。考虑到在 Java 代码中，我特意插入了一个新的、理论上未与其他表（如 Ticket 或 Orderinfo）建立关联记录的乘客 ID 来测试删除，这次成功删除符合预期。这说明在没有外键约束冲突的情况下，该删除存储过程能够有效地从数据库中移除指定记录。

此部分实验成功构建了 Java 客户端与 MySQL 数据库之间的通信桥梁，验证了通过 JDBC 调用数据库服务器端存储过程的可行性与功能完整性。实验中，多个类型的存储过程（包括查询、插入、修改、删除）均能按预期执行并正确处理参数与结果，展示了客户端对服务器端业务逻辑的有效调用能力。同时，实验也暴露出 JDBC 与 MySQL 之间在异常报告方面存在不一致的潜在问题，例如“PROCEDURE ... does not exist”的误报现象，提示实际开发中可能遇到的底层技术挑战。尽管如此，Java 端的异常处理机制与资源管理（如 try-catch 和 try-with-resources）的良好设计保障了程序的稳定性。

1.4 触发器的设计与实现

触发器是数据库在特定表上发生预定义事件（如 INSERT, UPDATE, DELETE）时自动执行的一段代码。本次实验旨在通过触发器实现对 Passenger 表数据变动的审计日志功能。

日志表的设计与创建：TriggerLog

为了存储触发器捕获到的数据变动信息，我首先创建了一个名为 TriggerLog 的日志表。该表包含以下关键字段：

- log_id INT AUTO_INCREMENT PRIMARY KEY: 日志记录的自增主键。
- table_name VARCHAR(50) NOT NULL: 发生变动的表的名称(在本例中为'Passenger')。
- operation VARCHAR(10) NOT NULL: 导致触发器执行的操作类型 ('INSERT', 'UPDATE', 'DELETE')。
- item_id VARCHAR(50): 发生变动的记录的主键或其他唯一标识 (对于 Passenger 表, 我记录其 id_card)。
- description TEXT: 对变动的详细描述, 可以包含新旧数据值, 以便追溯。
- log_time DATETIME DEFAULT CURRENT_TIMESTAMP: 记录变动发生的精确时间, 默认为当前时间戳。


```

1 CREATE TABLE TriggerLog (
2     log_id INT AUTO_INCREMENT PRIMARY KEY,
3     table_name VARCHAR(50) NOT NULL,
4     operation VARCHAR(10) NOT NULL,
5     item_id VARCHAR(50),
6     description TEXT,
7     log_time DATETIME DEFAULT CURRENT_TIMESTAMP
8 );

```

触发器的具体实现

我在 Passenger 表上创建了三个 AFTER 类型的触发器，分别对应 INSERT、UPDATE 和 DELETE 操作。这意味着触发器逻辑在数据操作成功完成后执行。

1. **插入触发器 (after_passenger_insert):** 当向 Passenger 表成功插入一条新记录后，此触发器被激活。它使用 MySQL 提供的 NEW 伪行 (pseudo-row) 来访问新插入行的数据。触发器体内的 INSERT INTO TriggerLog 语句将新乘客的 id_card、姓名和年龄等信息记录到日志表中。

```

1     CREATE TRIGGER after_passenger_insert
2     AFTER INSERT ON Passenger
3     FOR EACH ROW -- 对于每一行受影响的记录都执行一次触发器体
4     BEGIN
5         -- NEW 伪行 (pseudo-row) 包含了新插入的记录的数据
6         INSERT INTO TriggerLog (table_name, operation, item_id,
7             description)
8         VALUES (
9             'Passenger',
10            'INSERT',
11            NEW.id_card, -- 新记录的身份证号作为项目 ID
12            CONCAT('New_passenger_inserted.ID:', NEW.id_card,
13                ', Name:', NEW.passname, ', Age:', NEW.age)
14        );
15     END //

```

2. **更新触发器 (after_passenger_update):** 当 Passenger 表中的某条记录被成功更新后，此触发器被激活。它同时使用 OLD (代表更新前的数据) 和 NEW (代表更新后的数据) 伪行。触发器将更新操作记录到 TriggerLog，并在描述中指明了被更新记录的 ID 以及姓名和电话字段更新前后的值，以便清晰地了解变动内容。

```

1     CREATE TRIGGER after_passenger_update
2     AFTER UPDATE ON Passenger
3     FOR EACH ROW

```

```

4      BEGIN
5          -- OLD 伪行包含更新前的记录数据, NEW 伪行包含更新后的记录数据
6          INSERT INTO TriggerLog (table_name, operation, item_id,
7                                  description)
8              VALUES (
9                  'Passenger',
10                 'UPDATE',
11                 NEW.id_card, -- 更新后记录的身份证号作为项目 ID
12                 CONCAT('Passenger_updated.ID:', OLD.id_card, ', ',
13                         Name_from', OLD.passname, '"_updated_to"', NEW
14                         .passname, '"_Phone_from', OLD.phone, '"_
15                         updated_to"', NEW.phone, '"')
16             );
17      END //

```

3. **删除触发器 (after_passenger_delete):** 当从 Passenger 表中成功删除一条记录后, 此触发器被激活。它使用 OLD 伪行来访问被删除行的数据。触发器将被删除乘客的 id_card 和姓名等信息记录到日志表中。

```

1      CREATE TRIGGER after_passenger_delete
2      AFTER DELETE ON Passenger
3      FOR EACH ROW
4      BEGIN
5          -- OLD 伪行包含了被删除的记录的数据
6          INSERT INTO TriggerLog (table_name, operation, item_id,
7                                  description)
8              VALUES (
9                  'Passenger',
10                 'DELETE',
11                 OLD.id_card, -- 被删除记录的身份证号作为项目 ID
12                 CONCAT('Passenger_deleted.ID:', OLD.id_card, ', ',
13                         Name:', OLD.passname)
14             );
15      END //

```

所有触发器的定义同样使用了 DELIMITER // 命令。

1.5 触发器执行效果测试与分析

为了验证触发器的功能, 我直接在 MySQL 客户端 (如 MySQL Shell 或 Workbench) 对 Passenger 表执行 INSERT, UPDATE, 和 DELETE 操作, 并在每次操作后查询 TriggerLog 表以观察日志记录。

INSERT 操作触发测试结果

执行以下 SQL 插入一条新乘客记录：

Code Listing 1: 测试 INSERT 触发器的 SQL

```
1 INSERT INTO Passenger (id_card, passname, age, phone)
2 VALUES ('51010119950101123X', 'Wangping', 28, '13912345678');
3 SELECT * FROM TriggerLog WHERE table_name = 'Passenger' AND operation
   = 'INSERT' ORDER BY log_time DESC LIMIT 1;
```

查询 TriggerLog 表的结果应显示新记录，其中 operation 为'INSERT'，item_id 为'51010119950101123X'，description 包含新插入乘客的姓名和年龄。

log_id	table_name	operation	item_id	description	log_time
6	Passenger	INSERT	51010119950101123X	New passenger inserted. ID: 51010119950101123X, Name: Wangping, Age: 28	2025-05-25 20:09:14
3	Passenger	INSERT	110101200001018888	New passenger inserted. ID: 110101200001018888, Name: 待删除用户, Age: 30	2025-05-25 20:04:42
1	Passenger	INSERT	110101200001019999	New passenger inserted. ID: 110101200001019999, Name: 测试用户, Age: 25	2025-05-25 19:50:37

3 rows in set (0.01 sec)

图 8: Passenger 表 INSERT 操作后 TriggerLog 表查询结果

这证明了 after_passenger_insert 触发器在插入操作后被成功激活并正确记录了日志。

UPDATE 操作触发测试结果

执行以下 SQL 更新刚插入的乘客记录：

Code Listing 2: 测试 UPDATE 触发器的 SQL

```
1 UPDATE Passenger
2 SET phone = '13887654321', age = 29
3 WHERE id_card = '51010119950101123X';
4 SELECT * FROM TriggerLog WHERE table_name = 'Passenger' AND operation
   = 'UPDATE' ORDER BY log_time DESC LIMIT 1;
```

查询 TriggerLog 表的结果应显示新记录，其中 operation 为'UPDATE'，item_id 为'51010119950101123X'，description 应包含更新前后电话和姓名的变化。

log_id	table_name	operation	item_id	description	log_time
7	Passenger	UPDATE	51010119950101123X	Passenger updated. ID: 51010119950101123X, Name from "Wangping" updated to "Wangping", Phone from "13912345678" updated to "13887654321"	2025-05-25 20:09:14
4	Passenger	UPDATE	110101200001019999	Passenger updated. ID: 110101200001019999, Name from "测试用户" updated to "测试用户", Phone from "13900000000" updated to "13900000000"	2025-05-25 20:04:42
2	Passenger	UPDATE	110101200001019999	Passenger updated. ID: 110101200001019999, Name from "测试用户" updated to "测试用户", Phone from "13812345678" updated to "13900000000"	2025-05-25 19:50:37

3 rows in set (0.00 sec)

图 9: Passenger 表 UPDATE 操作后 TriggerLog 表查询结果

这证明了 after_passenger_update 触发器在更新操作后被成功激活，并能够访问 OLD 和 NEW 伪行来记录变动详情。

DELETE 操作触发测试结果

执行以下 SQL 删除该乘客记录（假设无外键约束阻止）：

Code Listing 3: 测试 DELETE 触发器的 SQL

```
1 DELETE FROM Passenger
2 WHERE id_card = '51010119950101123X';
3 SELECT * FROM TriggerLog WHERE table_name = 'Passenger' AND operation
   = 'DELETE' ORDER BY log_time DESC LIMIT 1;
```

查询 TriggerLog 表的结果应显示新记录，其中 operation 为'DELETE'，item_id 为'51010119950101123X'，description 包含被删除乘客的姓名。

log_id	table_name	operation	item_id	description	log_time
8	Passenger	DELETE	51010119950101123X	Passenger deleted. ID: 51010119950101123X, Name: Wangping	2025-05-25 20:09:14
5	Passenger	DELETE	110101200001018888	Passenger deleted. ID: 110101200001018888, Name: 待删除用户	2025-05-25 20:04:42

2 rows in set (0.00 sec)

图 10: Passenger 表 DELETE 操作后 TriggerLog 表查询结果

这证明了 after_passenger_delete 触发器在删除操作后被成功激活，并正确记录了被删除记录的信息。

所有触发器测试结果均符合预期，表明触发器能够可靠地响应数据库事件并执行预定义的日志记录任务。

2 题目 2：索引实验

题目要求：

1. 结合作业 3，针对你的数据库中的一个表，编写简单的数据查询（查询语句应包括单个涉及非主属性等值比较的查询条件，设该非主属性为 A ，具体属性结合业务背景）和数据插入语句，程序应能在终端或服务器以文件形式记录每次数据读写操作的耗时。
2. 无索引测试：执行查询（查询条件不包含主码，且不存在针对属性 A 建立的索引），记录不同数据规模下的查询时间，
3. 有索引测试：针对属性 A 建立索引，采用与 2 中相同的查询，记录不同数据规模下的查询时间。
4. 分析实验数据，制作图表，比较有索引和无索引的情况下，查询时间随数据量增加的变化情况，分析导致实验结果的原因。

这个实验主要是深入理解，数据库索引在提升数据查询性能方面的核心作用和具体影响。这里主要用到 pymysql 进行实验（实验代码见附录B）。

2.1 实验前准备

为了模拟不同负载下的查询性能，实验采用了动态生成测试数据的方法，主要针对 Passenger 表进行。

1. **基础数据样本 (BASE_PASSENGER_DATA):** 我预定义了一个包含 52 条乘客记录的 Python 列表作为基础数据。每条记录包含 `id_card`, `passname`, `age`, `phone`。
2. **数据规模扩展 (DATA_SCALES):** 实验设计了多个数据规模进行测试，分别为：1,000、10,000、50,000、100,000、500,000 和 1,000,000 条记录。
3. **数据插入策略与表清理:**

- 在每个数据规模测试开始前，使用 `TRUNCATE TABLE Passenger` 命令清空 Passenger 表，以保证测试环境的纯净。为避免因外键约束（尽管 Passenger 表本身没有外键，但作为好习惯）导致 `TRUNCATE` 失败，在操作前后分别执行了 `SET FOREIGN_KEY_CHECKS = 0;` 和 `SET FOREIGN_KEY_CHECKS = 1;`。
- 通过重复基础数据样本并动态生成唯一的 `id_card`（采用“P”+时间戳后6位+补零序号的策略，确保长度在 `VARCHAR(20)` 内且唯一）来填充 Passenger 表至目标规模。
- 数据插入采用批量方式（每批 2000 条），并禁用 MySQL 的自动提交，在每批插入完成后手动提交事务（`conn.commit()`），以提高大量数据插入的效率。

实验的核心是测试针对非主属性的等值比较查询。我选择 Passenger 表中的 `passname` 字段作为目标查询属性（即作业要求的非主属性 A）。查询语句的 SQL 模板为：

Code Listing 4: 实验核心查询语句

```
1 SELECT * FROM Passenger WHERE passname = %s;
```

在 Python 脚本中，`%s` 作为参数占位符，实际执行时会被替换为目标姓名 `TARGET_NAME`（实验中设置为“Matthew”）。我选择查询所有列（`SELECT *`）是为了模拟一个常见的需要回表操作的场景。

实验的关键在于对比有无索引的情况：

1. **无索引测试:** 在进行该部分测试前，程序会尝试删除名为 `idx_passenger_passname` 的索引（如果存在），以确保查询是在 `passname` 字段没有任何显式索引的情况下进行的。
2. **有索引测试:** 在进行该部分测试前，程序会在 Passenger 表的 `passname` 字段上创建一个名为 `idx_passenger_passname` 的 B-Tree 索引，SQL 语句为 `CREATE INDEX idx_passenger_passname ON Passenger (passname);`。

2.2 性能记录与测试流程

为了得到可靠的性能数据，实验遵循以下流程：

1. **计时方法与精度：**使用 Python 的 `time.perf_counter()` 函数，该函数提供高精度的计时，能够捕捉到毫秒甚至微秒级别的差异。查询耗时被记录为毫秒 (ms)。
2. **多次查询与平均化：**针对每一个数据规模和索引状态（有或无），相同的查询语句会连续执行 `NUMBER_OF_QUERIES_PER_SCALE` 次（实验中设置为 30 次）。这样做的目的是为了平滑单次查询可能存在的系统抖动或缓存效应带来的偶然性误差，取其平均查询耗时作为该条件下的性能指标。
3. **结果记录与输出：**每次完成一个数据规模下的有/无索引测试后，会将数据规模、索引是否存在（布尔值）以及计算得到的平均查询时间（毫秒）追加记录到名为 `query_performance.csv` 的 CSV 文件中。CSV 文件在实验开始前会清空并写入表头。

整体实验流程如图11：

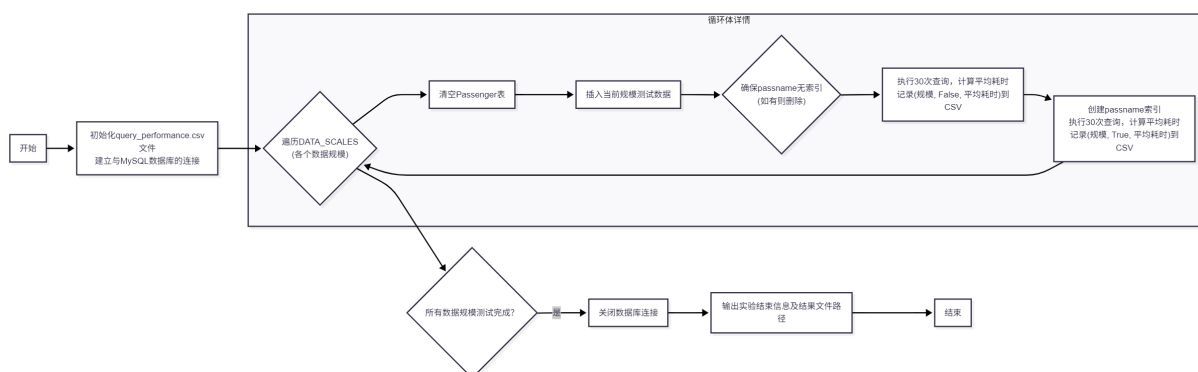


图 11: 实验流程图

2.3 实验结果与数据分析

实验执行后，`query_performance.csv` 文件记录了不同数据规模下，有无索引时查询目标姓名“Matthew”的平均耗时。原始数据如下表所示：

表 1: 索引实验原始数据记录

DataScale (记录数)	IndexExists (索引是否存在)	QueryTimeMillis (平均查询时间/ms)
1000	False	0.773
1000	True	0.713
10000	False	4.448
10000	True	4.544
50000	False	21.228
50000	True	21.302
100000	False	30.716
100000	True	31.295
500000	False	133.608
500000	True	89.455
1000000	False	252.447
1000000	True	172.841

为了更直观地分析数据，对实验结果进行可视化，如图12与图13。

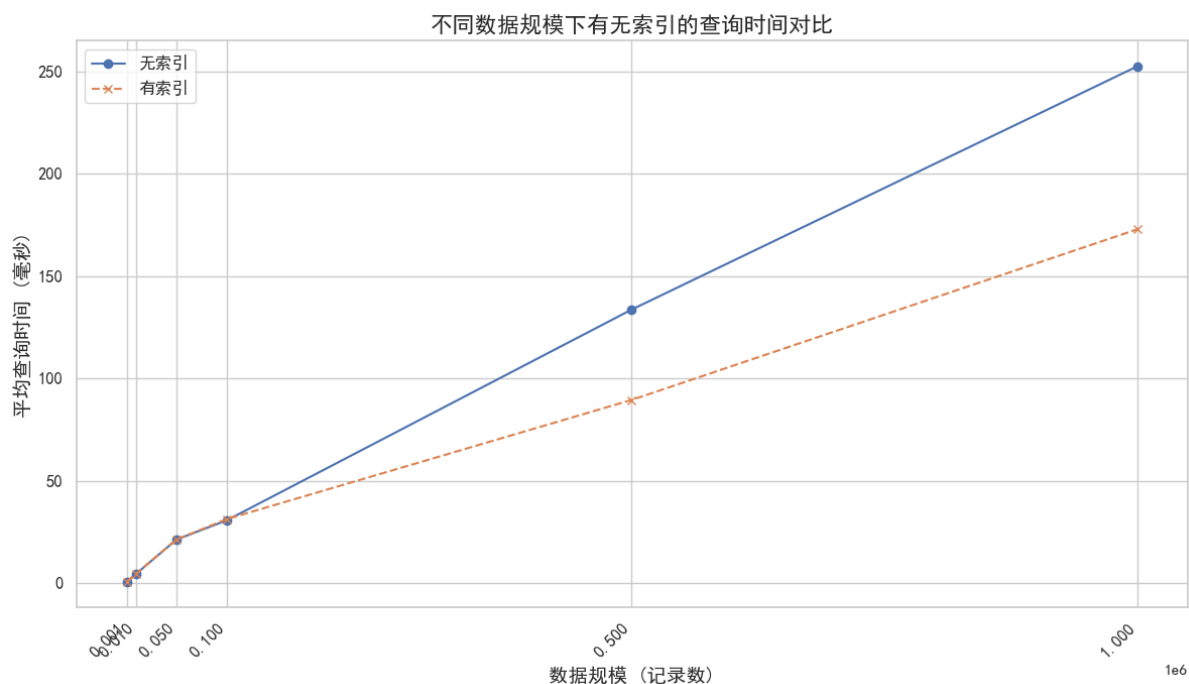


图 12: 不同数据规模下有无索引查询时间对比 (折线图)

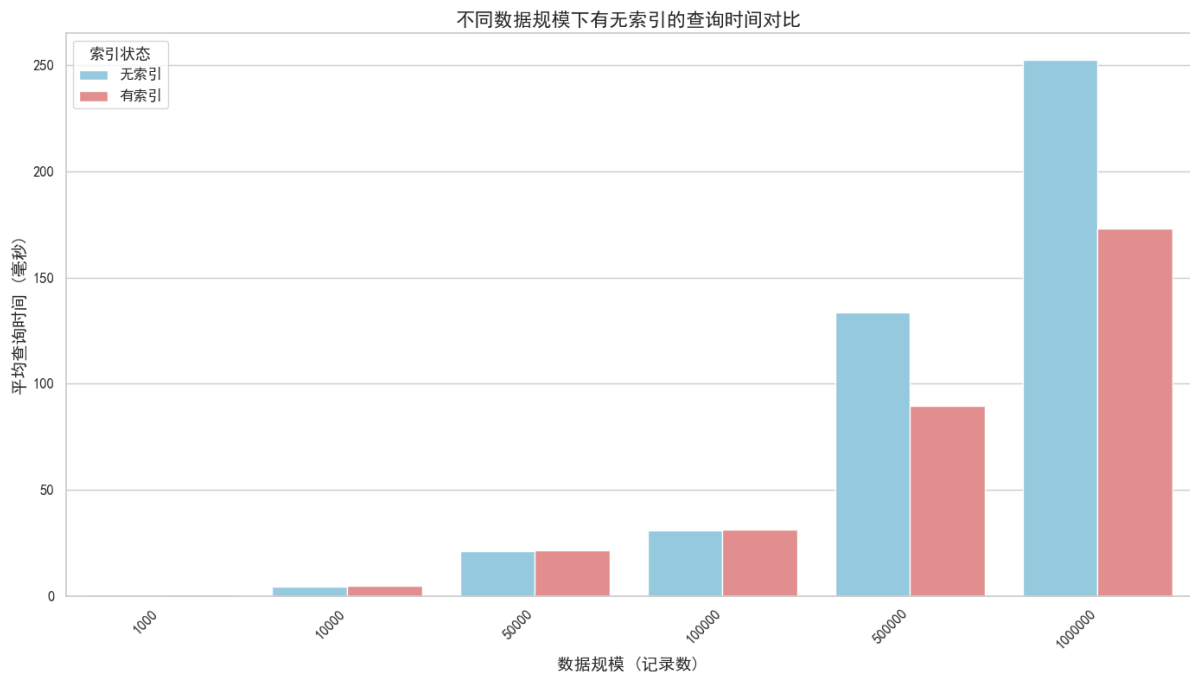


图 13: 不同数据规模下有无索引查询时间对比（柱状图）

通过折线图（图12）和柱状图（图13）可以清晰地看到在不同数据规模下，有索引和无索引查询的平均耗时对比。

从图12和图13中可以观察到：

- **小数据规模（1 千至 10 万条记录）：**在数据量较少时，有索引和无索引的查询时间差异非常小，几乎可以忽略不计。甚至在某些点（如 1 万、5 万、10 万条数据），有索引的查询时间略高于无索引的情况。这可能是因为数据量不大时，全表扫描的成本与通过索引查找并回表的成本相差无几，而索引本身也需要占用 I/O 资源进行读取和维护，优化器也可能因数据量小而选择全表扫描。
- **大数据规模（50 万至 100 万条记录）：**当数据量显著增加后，索引的优势开始凸显。在 50 万条记录时，无索引查询耗时约 133.6ms，有索引查询耗时约 89.5ms。在 100 万条记录时，无索引查询耗时约 252.4ms，而有索引查询耗时大幅降低至约 172.8ms。
- **增长趋势：**
 - **无索引：**查询时间随着数据规模的增加呈现出近乎线性的增长趋势。这是因为数据库必须扫描表中的每一行来找到匹配的记录。
 - **有索引：**在数据量较大时，有索引查询时间的增长速度明显放缓，远低于无索引的情况。这符合 B-Tree 索引查找时间复杂度 $O(\log N)$ 的特性。

为了量化索引带来的性能提升，我计算了在不同数据规模下，使用索引相对于无索引查询时间的提升百分比（图14）。计算公式为：

$$\text{性能提升百分比} = \frac{(\text{无索引查询时间} - \text{有索引查询时间})}{\text{无索引查询时间}} \times 100\%$$

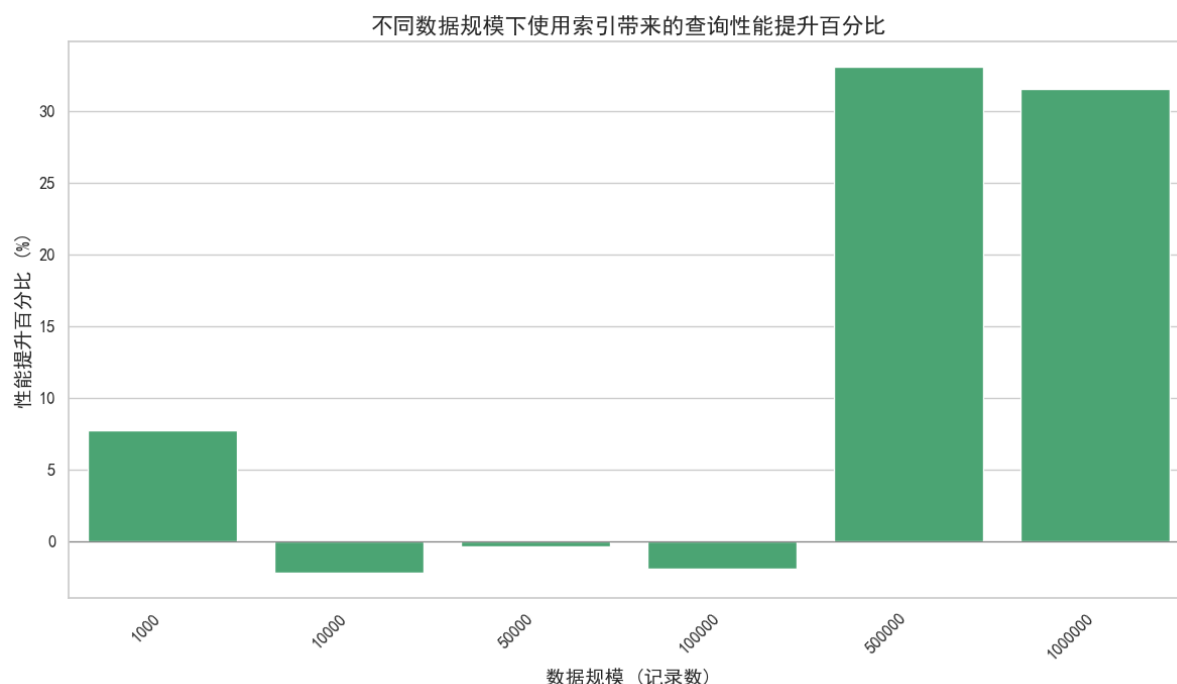


图 14: 不同数据规模下使用索引带来的查询性能提升百分比

从图14中可以观察到，在数据规模达到 50 万时，索引带来了约 33% 的性能提升；在 100 万数据规模时，性能提升约 31.5%。这清晰地展示了索引在大数据量下的优化效果。在小数据量时，性能提升百分比可能为负或接近于零，与前述分析一致。

2.4 实验结果原因分析

实验结果清晰地反映了索引对查询性能的影响，其根本原因在于数据库执行查询时采用的不同数据访问策略：

- **无索引查询 (全表扫描 - Full Table Scan):** 当查询的 WHERE 子句中引用的列（本例中为 `passname`）没有索引时，数据库引擎无法快速定位到包含特定值的记录。它唯一的选择是逐行读取表中的所有数据页，检查每一行的 `passname` 字段是否与目标值（"Matthew"）匹配。这种操作的 I/O 开销和 CPU 开销与表中的总行数成正比，因此当数据量增大时，查询时间会显著增加。
- **有索引查询** 当在 `passname` 字段上创建了 B-Tree 索引后，数据库引擎可以利用这个有序的索引结构来高效地查找。
 1. **索引定位:** 数据库首先在 `passname` 的 B-Tree 索引中查找值为 "Matthew" 的条目。B-Tree 的查找效率非常高，通常只需要几次磁盘 I/O 就能定位到索引叶子节点。
 2. **获取行指针:** 索引的叶子节点不仅存储了索引键值 ("Matthew")，还存储了指向包含这些键值的数据行在表中的物理位置的指针（如 RowID 或主键值，取决于索引类型和存储引擎）。

3. **回表查询 (Table Access by RowID/Primary Key)**: 由于我的查询是 `SELECT *`, 需要获取匹配行的所有列数据, 而这些数据并非全部存储在 `passname` 的非聚集索引中。因此, 数据库在通过索引找到匹配记录的指针后, 还需要根据这些指针去数据表中读取完整的行数据。

尽管存在回表操作, 但在数据量较大且索引选择性较好 (即目标值在表中出现的次数相对较少) 的情况下, 通过索引定位少数几行再进行回表的总成本, 远低于扫描整个表的成本。

因此通过实验结果, 我可以总结, 对于非常小的数据表, 全表扫描可能直接在内存中完成, 其速度非常快。此时, 使用索引可能涉及到额外的索引 I/O 以及优化器进行索引选择的判断开销, 反而可能导致轻微的性能下降或差异不明显。

并且, 索引的效果也与查询条件中值的选择性密切相关。如果查询的值在表中非常普遍 (选择性低), 即使有索引, 数据库也可能需要回表读取大量数据行, 此时索引的优势会减弱, 优化器甚至可能放弃使用索引。在本实验中, "Matthew" 在基础样本中出现了两次, 在扩展数据后, 其出现的频率相对较低, 这有利于索引发挥作用。

2.5 实验结论

1. **索引是提升大数据量下查询性能的关键手段**: 实验数据表明, 在数据规模达到数十万乃至百万级别时, 为查询条件中涉及的非主属性列创建索引, 能够显著降低查询响应时间。但是在数据量较小的情况下, 有无索引对本实验中的等值查询性能影响不显著, 甚至可能因额外开销而略微降低性能。索引的优势主要体现在大数据量场景。
2. **无索引查询的线性扩展性问题**: 缺乏索引的查询通常会导致全表扫描, 其查询时间随数据量的增加而近似线性增长, 这在大型数据库中是不可接受的。
3. **索引提升性能的原理**: B-Tree 等索引结构通过提供有序的数据访问路径, 使得数据库能够快速定位到满足查询条件的记录, 避免了对整个数据表的扫描, 从而大幅减少了 I/O 操作和 CPU 计算量。

因此, 虽然索引能提升查询性能, 但创建和维护索引本身也需要占用磁盘空间, 并会在数据插入、更新和删除时产生额外的开销。因此, 索引的设计需要权衡查询性能的提升与 DML 操作性能的潜在影响以及存储成本。

3 总结

本次实验对数据库系统的核心性能与自动化机制进行了探究, 涵盖了存储过程、触发器以及索引这三大关键领域。从前端 Java 应用程序通过 JDBC 调用服务器端存储过程, 到后台数据库自动响应数据变动, 再到通过索引优化查询性能, 整个实验流程让我对数据库的内部运作及其与外部应用的协同有了更为深刻的理解。我不仅成功地实现了各项功能, 也在此过程中直面了理论与实践相结合时的真实挑战。

具体而言，通过实现对航班信息的查询、乘客的增删改等操作，我清晰地看到存储过程如何将数据库操作模块化。同时，在数据审计场景下，触发器则展现了其自动化响应数据变动的强大能力，确保了数据完整性和日志记录的实时性。

此外，索引实验有力地证明了索引在大规模数据查询中的决定性作用，尤其是在数十万乃至百万级别的数据量下，索引对非主属性查询的性能提升是显著的。这进一步强化了我对 B-Tree 索引工作原理的认识，理解了其如何避免低效的全表扫描。与此同时，实验也揭示了在小数据量下，索引可能带来的额外开销和优化器决策的微妙之处，这提示我索引并非万能药，需根据具体的数据规模和查询模式进行审慎选择和优化。这些实践经验为我学习数据库相关知识打下了实践基础。

A 附录 A: JDBC 存储过程调用

Code Listing 5: JDBC 存储过程调用

```
1 import java.sql.*;
2
3 // 存储过程调用器类
4 public class SPCaller { // 公共类必须与文件名相同 (SPCaller.java)
5
6     // 数据库连接参数
7     // Ensure your MySQL server is running and 'airlinedb' database exists
8     // Added timezone parameter for newer JDBC versions to avoid warnings/errors
9     private static final String DB_URL = "jdbc:mysql://localhost:3306/airlinedb?serverTimezone=UTC";
10    private static final String USER = "root";
11    private static final String PASS = "XXXXXX"; // 密码
12
13    // main 方法: Java 程序的入口点
14    public static void main(String[] args) {
15        System.out.println("程序开始...");
16        System.out.println("尝试连接到数据库...");
17
18        // 使用 try-with-resources 块自动管理 Connection 对象, 确保连接在使用后关闭
19        try (Connection conn = getConnection()) {
20            System.out.println("数据库连接成功!");
21
22            // --- 调用查询存储过程 ---
23            System.out.println("\n---调用GetFlightsByRoute存储过程(查询)---");
24            callGetFlightsByRoute(conn, "PEK", "PVG");
25            callGetFlightsByRoute(conn, "ABC", "XYZ");
26
27            // --- 调用数据插入存储过程 ---
28            System.out.println("\n---调用AddPassenger存储过程(插入)---");
29            String testIdCard = "110101200001019999"; // 示例乘客ID, 用于测试
30            String testIdCardNew = "110101200001018888"; // 一个新的, 用于删除测试的ID
```

```

31
32 // 尝试插入一个已存在的乘客（假设 testIdCard 已存在，会报
    错）
33 System.out.println("尝试插入已存在的乘客（预期错误）：");
34 addPassenger(conn, testIdCard, "测试用户-已存在", 25, "
    13812345678");
35
36 // 插入一个新的乘客，用于后续删除测试
37 System.out.println("\n插入一个新的乘客用于删除测试：");
38 addPassenger(conn, testIdCardNew, "待删除用户", 30, "
    13987654321");
39
40 // --- 调用数据修改存储过程 ---
41 System.out.println("\n---调用UpdatePassengerPhone存储
    过程（修改）---");
42 // 更新之前已存在的乘客的电话（假设 testIdCard 对应的乘客
    存在）
43 updatePassengerPhone(conn, testIdCard, "13900000000");
44 // 尝试更新不存在的乘客ID，观察输出信息
45 updatePassengerPhone(conn, "9999999999999999999", "
    13012345678");
46
47
48 // --- 调用数据删除存储过程 ---
49 System.out.println("\n---调用DeletePassenger存储过程（
    删除）---");
50
51 // 我将尝试删除上面新插入的 testIdCardNew，它应该没有关联
    记录
52 deletePassenger(conn, testIdCardNew);
53
54 // 您也可以尝试删除 testIdCard，如果它有关联记录且没有级
    联删除，将会报错
55 // System.out.println("\n尝试删除可能有关联记录的乘客（可
    能会因外键约束失败）：");
56 // deletePassenger(conn, testIdCard);
57
58
59 } catch (SQLException e) {
60     // 捕获数据库操作过程中可能发生的 SQLException
61     System.err.println("数据库操作失败：" + e.getMessage());

```

```

62         e.printStackTrace(); // 打印详细的堆栈跟踪信息，有助于调
           试
63     } catch (Exception e) {
64         // 捕获其他意外的异常
65         System.err.println("发生意外错误:␣" + e.getMessage());
66         e.printStackTrace();
67     }
68
69     System.out.println("\n程序结束.");
70 }
71
72 // 方法：建立与数据库的连接
73 // 使用 DriverManager.getConnection 方法
74 private static Connection getConnection() throws SQLException {
75     // DriverManager 会根据 DB_URL 自动找到并加载合适的 JDBC 驱动
76     // 返回一个 Connection 对象
77     return DriverManager.getConnection(DB_URL, USER, PASS);
78 }
79
80 // --- 调用特定存储过程的方法 ---
81 // 这些方法负责构建 CallableStatement，设置参数，执行存储过程并处
   理结果
82
83 // 方法：调用 GetFlightsByRoute 存储过程（查询操作）
84 // 使用 CallableStatement 执行 { call GetFlightsByRoute(?, ?) }
85 public static void callGetFlightsByRoute(Connection conn, String
   depAirport, String arrAirport) {
86     if (conn == null) return; // 检查连接是否有效
87
88     System.out.println("调用␣GetFlightsByRoute␣for:␣" +
           depAirport + "␣to␣" + arrAirport);
89     String callSQL = "{␣call␣GetFlightsByRoute(?,␣?)␣}"; // JDBC
   调用语法
90
91     try (CallableStatement cstmt = conn.prepareCall(callSQL)) {
92         // 创建 CallableStatement
93
94         // 设置输入参数（参数索引从 1 开始对应存储过程定义中的参
           数顺序）
95         cstmt.setString(1, depAirport);
           cstmt.setString(2, arrAirport);

```

```

96
97 // 执行存储过程。execute() 返回 true 表示返回了结果集，否
    则返回 false。
98 boolean hasResultSet = pstmt.execute();
99
100 // 处理结果集（如果存储过程返回一个结果集）
101 if (hasResultSet) {
102     try (ResultSet rs = pstmt.getResultSet()) { // 获取结
        果集
103         System.out.println("找到的航班:");
104         // 打印结果集的列头部
105         ResultSetMetaData metaData = rs.getMetaData();
106         int columnCount = metaData.getColumnCount();
107         for (int i = 1; i <= columnCount; i++) {
108             System.out.printf("%-25s", metaData.
                getColumnLabel(i)); // 使用 getColumnLabel
                获取列名或别名
109         }
110         System.out.println();
111         for (int i = 0; i < columnCount; i++) {
112             System.out.print("-----")
                ;
113         }
114         System.out.println();
115
116         while (rs.next()) { // 遍历每一行结果
117             // 按列名检索数据
118             String flightNumber = rs.getString("
                flight_number");
119             String airModel = rs.getString("air_model");
120             Timestamp depTime = rs.getTimestamp("
                departure_time");
121             Timestamp arrTime = rs.getTimestamp("
                arrival_time");
122             String depName = rs.getString("
                departure_airport_name");
123             String depCity = rs.getString("departure_city
                ");
124             String arrName = rs.getString("
                arrival_airport_name");
125             String arrCity = rs.getString("arrival_city")

```

```

126         ;
        String airlineName = rs.getString("
            airline_name");

127
128         // 打印检索到的数据
129         System.out.printf("%-25s", flightNumber);
130         System.out.printf("%-25s", airModel);
131         System.out.printf("%-25s", depTime);
132         System.out.printf("%-25s", arrTime);
133         System.out.printf("%-25s", depName);
134         System.out.printf("%-25s", depCity);
135         System.out.printf("%-25s", arrName);
136         System.out.printf("%-25s", arrCity);
137         System.out.printf("%-25s", airlineName);
138         System.out.println(); // 打印完一行数据后换行
139     }
140 }
141 } else {
142     System.out.println("此查询没有返回结果集。");
143 }
144
145 } catch (SQLException e) {
146     System.err.println("调用GetFlightsByRoute存储过程时出错
        : " + e.getMessage());
147     // e.printStackTrace(); // Uncomment for full stack trace
148 }
149 }
150
151 // 方法：调用 AddPassenger 存储过程（插入操作）
152 // 使用 CallableStatement 执行 { call AddPassenger(?, ?, ?, ?) }
153 public static void addPassenger(Connection conn, String idCard,
    String name, int age, String phone) {
154     if (conn == null) return; // 检查连接是否有效
155
156     System.out.println("正在为：" + name + "(" + idCard + ")调用
        AddPassenger存储过程");
157     String callSQL = "{callAddPassenger(?,?,?,?)}";
158
159     try (CallableStatement cstmt = conn.prepareCall(callSQL)) {
160         // 创建 CallableStatement

```



```

161         // 设置输入参数
162         pstmt.setString(1, idCard);
163         pstmt.setString(2, name);
164         pstmt.setInt(3, age);
165         pstmt.setString(4, phone);
166
167         // 执行存储过程。executeUpdate() 返回受影响的行数。
168         int rowsAffected = pstmt.executeUpdate();
169
170         if (rowsAffected > 0) {
171             System.out.println("AddPassenger调用成功。受影响行数
172             : " + rowsAffected);
173         } else {
174             System.out.println("AddPassenger存储过程已执行，但没
175             有行受到影响（可能指示插入失败或重复）。");
176         }
177
178     } catch (SQLException e) {
179         System.err.println("调用AddPassenger存储过程时出错：" +
180         e.getMessage());
181         // e.printStackTrace(); // 取消注释以获取完整的堆栈跟踪
182     }
183
184     // 方法：调用 DeletePassenger 存储过程（删除操作）
185     // 使用 CallableStatement 执行 { call DeletePassenger(?) }
186     public static void deletePassenger(Connection conn, String idCard
187     ) {
188         if (conn == null) return; // 检查连接是否有效
189
190         System.out.println("正在为ID:" + idCard + "调用
191         DeletePassenger存储过程");
192         String callSQL = "{callDeletePassenger(?)}";
193
194         try (CallableStatement pstmt = conn.prepareCall(callSQL)) {
195             // 创建 CallableStatement
196
197             // 设置输入参数
198             pstmt.setString(1, idCard);
199
200             // 执行 DELETE 操作的存储过程

```

```

196         int rowsAffected = pstmt.executeUpdate();
197
198         if (rowsAffected > 0) {
199             System.out.println("DeletePassenger调用成功。受影响
200                 行数:" + rowsAffected);
201         } else {
202             System.out.println("DeletePassenger存储过程已执行,
203                 但没有行受到影响。乘客ID'" + idCard + "'可能不
204                 存在或已被删除。");
205         }
206
207     } catch (SQLException e) {
208         System.err.println("调用DeletePassenger存储过程时出错:
209             " + e.getMessage());
210         // e.printStackTrace(); // 取消注释以获取完整的堆栈跟踪
211     }
212 }
213
214 // 方法: 调用 UpdatePassengerPhone 存储过程 (修改操作)
215 // 使用 CallableStatement 执行 { call UpdatePassengerPhone(?, ?)
216 }
217
218 public static void updatePassengerPhone(Connection conn, String
219     idCard, String newPhone) {
220     if (conn == null) return; // 检查连接是否有效
221
222     System.out.println("正在为ID:" + idCard + "调用
223         UpdatePassengerPhone存储过程, 新电话为:" + newPhone);
224     String callSQL = "{callUpdatePassengerPhone(?,?)}";
225
226     try (CallableStatement pstmt = conn.prepareCall(callSQL)) {
227         // 创建 CallableStatement
228
229         // 设置输入参数
230         pstmt.setString(1, idCard);
231         pstmt.setString(2, newPhone);
232
233         // 执行 UPDATE 操作的存储过程
234         int rowsAffected = pstmt.executeUpdate();
235
236         if (rowsAffected > 0) {
237             System.out.println("UpdatePassengerPhone调用成功。受

```

```
        影响行数:" + rowsAffected);
229     } else {
230         System.out.println("UpdatePassengerPhone_存储过程已执
            行, 但没有行受到影响。乘客_ID_" + idCard + "'_可能
            不存在或电话号码已是最新。");
231     }
232
233     } catch (SQLException e) {
234         System.err.println("调用_UpdatePassengerPhone_存储过程时
            出错:" + e.getMessage());
235         // e.printStackTrace(); // 取消注释以获取完整的堆栈跟踪
236     }
237 }
238 }
```

B 附录 B: Python 实验脚本

Code Listing 6: Python 实验脚本

```
1 import pymysql
2 import time
3 import csv
4 import random
5 import string
6 import sys
7
8 # --- 数据库连接参数 ---
9 DB_HOST = 'localhost'
10 DB_PORT = 3306
11 DB_USER = 'root'
12 DB_PASSWORD = 'xxxxxx' # 密码
13 DB_NAME = 'airlinedb'
14
15 # --- 实验配置 ---
16 RESULT_FILE = 'query_performance.csv' # 结果记录文件路径
17 # 定义需要测试的数据规模 (行数) 1千, 1万, 5万, 10万, 50万, 100万
18 DATA_SCALES = [1000, 10000, 50000, 100000, 500000, 1000000]
19 TARGET_NAME = "Matthew"
20 NUMBER_OF_QUERIES_PER_SCALE = 30
21
22 # --- 基础 Passenger 数据样本 ---
23 BASE_PASSENGER_DATA = [
24     ('110101199001011234', 'Zhangsan', 30, '13800138000'),
25     ('110101199002022345', 'Lisi', 28, '13900139000'),
26     ('110101198810243608', 'Sarah', 31, '+1-009-486-7233x49762'),
27     ('110101196803223943', 'Becky', 41, '928.360.5914x489'),
28     ('110101197507058250', 'Erin', 42, '267.199.0167x98313'),
29     ('110101198411164376', 'Lindsey', 39, '(247)989-2111x256'),
30     ('110101195011276882', 'Rachel', 50, '(695)853-1407x0976'),
31     ('110101196909127547', 'Scott', 37, '001-153-608-6315'),
32     ('110101198610289070', 'Shirley', 50, '(132)996-0514'),
33     ('110101199506209367', 'Matthew', 34, '339.628.9836x7628'),
34     ('110101198709081849', 'Joshua', 50, '643.108.8938'),
35     ('110101199103086642', 'James', 46, '428.095.5648'),
36     ('110101199806282974', 'Erin', 63, '997-946-5506x277'),
37     ('110101199602114217', 'Hayley', 44, '309-454-1411x140'),
38     ('110101198909015468', 'Jamie', 62, '(032)027-5531x1870'),
```

```

39 ('110101199806092967', 'Spencer', 69, '001-214-217-8565x2501'),
40 ('110101195710268104', 'Matthew', 57, '3250111485'),
41 ('110101196205083417', 'Jillian', 62, '001-037-770-3037x77578'),
42 ('110101198908252802', 'Adam', 70, '001-674-825-7096'),
43 ('110101195902078356', 'Angelica', 39, '671.252.2284x988'),
44 ('110101199407095228', 'Henry', 51, '(484)693-2230x43443'),
45 ('110101197702183195', 'Eric', 25, '(955)463-7007x91646'),
46 ('110101197212145297', 'Jonathon', 54, '001-403-902-1744x8060'),
47 ('110101195010215457', 'Robin', 42, '2130427306'),
48 ('110101199407142760', 'Angela', 30, '191-603-1317'),
49 ('110101199606058320', 'Cory', 65, '199-279-8019x342'),
50 ('110101198910026821', 'Cindy', 68, '057.053.6953x57258'),
51 ('110101197404029144', 'Cynthia', 65, '8588019068'),
52 ('110101199806024063', 'Dorothy', 52, '815.165.9534x0806'),
53 ('110101195405286187', 'Mary', 70, '572.245.9752x801'),
54 ('110101199805127060', 'Brandon', 56, '+1-306-591-1181x8776'),
55 ('110101196407104603', 'Anthony', 54, '001-461-627-5292'),
56 ('110101198610116271', 'Chris', 22, '6369459526'),
57 ('110101196009264879', 'Candice', 59, '+1-865-973-5344x0720'),
58 ('110101197401175503', 'David', 28, '843.438.0606'),
59 ('110101196010073093', 'Amber', 38, '724.764.6974x6847'),
60 ('110101198208177794', 'Monica', 69, '(759)453-6726x076'),
61 ('110101197011228296', 'Alexander', 43, '011.380.8012x255'),
62 ('110101195603061279', 'Patrick', 42, '(746)521-7149x5387'),
63 ('110101196206102759', 'Belinda', 41, '+1-922-270-8375x211'),
64 ('110101197204248910', 'Christopher', 53, '+1-242-372-4582'),
65 ('110101198306273248', 'Earl', 23, '257-651-7307'),
66 ('110101198312118306', 'Mallory', 40, '1729969372'),
67 ('110101195510276691', 'Sheryl', 68, '042.636.5506'),
68 ('110101198908196586', 'Sherry', 22, '(957)088-6953'),
69 ('110101197607234259', 'Cynthia', 49, '001-136-356-1089'),
70 ('110101198502083115', 'Cameron', 67, '881-408-4736x2532'),
71 ('110101195510198228', 'Kimberly', 28, '639.453.5941x8088'),
72 ('110101197906032146', 'Michael', 69, '(774)996-3806'),
73 ('110101199611172474', 'Crystal', 38, '(884)917-4311x866'),
74 ('110101197705159207', 'Tara', 30, '(906)247-7268x7692'),
75 ('110101198603093949', 'Jared', 39, '001-975-440-0365x55372')

```

```

76 ]
77 # 样本数据集中Passenger记录总数
78 BASE_DATA_SIZE = len(BASE_PASSENGER_DATA)
79

```

```

80
81 # --- 数据库操作函数 ---
82 def get_connection():
83     """获取数据库连接"""
84     try:
85         conn = pymysql.connect(
86             host=DB_HOST,
87             port=DB_PORT,
88             user=DB_USER,
89             password=DB_PASSWORD,
90             database=DB_NAME,
91             charset='utf8mb4',
92             cursorclass=pymysql.cursors.DictCursor # 使用字典游标,
               结果以字典形式返回
93         )
94         return conn
95     except pymysql.MySQLError as e:
96         print(f"数据库连接失败: {e}", file=sys.stderr)
97         return None
98
99
100 def execute_sql(conn, sql, params=None):
101     """
102     执行单个 SQL 语句（不返回结果集），失败时抛出 pymysql.MySQLError
        异常。
103     """
104     if not conn:
105         raise pymysql.MySQLError(
106             "Database connection is None, cannot execute SQL.")
107     try:
108         with conn.cursor() as cursor:
109             cursor.execute(sql, params)
110             conn.commit()
111             return True
112     except pymysql.MySQLError as e:
113         raise e
114
115 # --- 实验数据准备函数 ---
116
117
118 def truncate_passenger_table(conn):

```

```

119     """
120     清空 Passenger 表，临时禁用外键检查以解决引用问题。返回 True 表示
        成功，False 表示失败。
121     """
122     print("正在清空 Passenger 表...")
123     if not conn:
124         return False
125
126     try:
127         with conn.cursor() as cursor:
128             cursor.execute("SET FOREIGN_KEY_CHECKS=0;") # 临时禁用
                外键约束检查
129             conn.commit()
130
131             # 执行 TRUNCATE TRUNCATE会清空表中的所有行，但表结构及其
                约束、索引等保持不变
132             cursor.execute("TRUNCATE TABLE Passenger")
133             conn.commit()
134
135             # 重新启用外键约束检查
136             cursor.execute("SET FOREIGN_KEY_CHECKS=1;")
137             conn.commit()
138
139             print("Passenger 表已清空.")
140             return True
141     except pymysql.MySQLError as e:
142         print(f"清空 Passenger 表失败:{e}", file=sys.stderr)
143         try: # 失败，但是也要尝试重新启用外键检查，防止数据库状态异
            常
144             with conn.cursor() as cursor:
145                 cursor.execute("SET FOREIGN_KEY_CHECKS=1;")
146                 conn.commit()
147         except pymysql.MySQLError as cleanup_e:
148             print(
149                 f"清空失败后重新启用 FOREIGN_KEY_CHECKS 失败:{
                    cleanup_e}", file=sys.stderr)
150             return False
151
152
153 def insert_scaled_passenger_data(conn, count, base_data):
154     """

```

```

155 通过重复基础数据样本并修改主键来插入指定数量的 Passenger 数据。返
    回 True 表示成功, False 表示失败。
156 """
157 print(f"正在插入_{count}_条_Passenger_数据_(基于_{BASE_DATA_SIZE}_
    _条样本)...")
158 if not conn:
159     return False
160
161 sql = "INSERT INTO Passenger(id_card,passname,age,phone)
    VALUES(%s,%s,%s,%s)"
162 data_to_insert = []
163
164 # 改进的 ID 生成策略: 确保唯一性和长度限制
165 # 策略: 'P' + (时间戳的后几位) + (当前序号, 补齐到足够位数)
166 # 例如: P + 123456 + 0000001 (14位, 在 VARCHAR(20) 范围内)
167 timestamp_part_for_id = str(int(time.time() * 1000) %
168                               1000000).zfill(6) # 时间戳后6位, 补0
169
170 # 序号部分需要足够的位数来容纳最大规模 (1,000,000 需要 7 位)
171 counter_padding_length = len(str(DATA_SCALES[-1])) # 确保能覆盖
    最大规模的序号位数
172
173 for i in range(count):
174     base_record_index = i % BASE_DATA_SIZE
175     original_id, name, age, phone = base_data[base_record_index]
176
177     # 组合生成新的唯一 id_card
178     new_id_card = f"P{timestamp_part_for_id}{str(i).zfill(
        counter_padding_length)}"
179
180     # 确保最终 ID 不超过 VARCHAR(20) 的限制
181     # 新策略下总长度为 1 (P) + 6 (timestamp) +
        counter_padding_length (max 7) = 14 或更少, 满足要求
182     if len(new_id_card) > 20: # 理论上不应该发生, 但作为安全检查
183         new_id_card = new_id_card[:20]
184
185     data_to_insert.append((new_id_card, name, age, phone))
186
187 # 批量插入并分批提交
188 BATCH_SIZE = 2000 # 每批插入的行数
189 start_time = time.perf_counter()

```



```

190
191 try:
192     with conn.cursor() as cursor:
193         conn.autocommit(False) # 禁用自动提交，以提高插入效率
194
195         for i in range(0, count, BATCH_SIZE):
196             batch = data_to_insert[i: i + BATCH_SIZE]
197             if not batch: # 防止最后一批为空
198                 break
199
200             cursor.executemany(sql, batch)
201             conn.commit() # 提交当前批次的事务
202
203             end_time = time.perf_counter()
204             print(f"数据插入完成，耗时：{(end_time - start_time) * 1000:.3f}ms")
205
206             return True
207 except pymysql.MySQLError as e:
208     print(f"插入数据失败：{e}", file=sys.stderr)
209     conn.rollback()
210     return False # 返回 False 表示插入失败
211 finally:
212     conn.autocommit(True) # 恢复自动提交为 True
213
214 # --- 索引管理函数 ---
215 def create_passenger_name_index(conn):
216     """创建 Passenger 表 passname 字段上的索引"""
217     index_name = "idx_passenger_passname"
218     print(f"正在创建索引 {index_name}...")
219     if not conn:
220         return False
221
222     try: # 先尝试删除，忽略不存在的错误
223         execute_sql(conn, f"DROP INDEX {index_name} ON Passenger")
224         print(f"旧索引 {index_name} 已删除 (如果存在).")
225     except pymysql.MySQLError as e:
226         if e.args[0] == 1091:
227             print(f"索引 {index_name} 不存在，无需删除.")
228         else:
229             print(f"删除索引时发生意外错误：{e}", file=sys.stderr)

```

```

230     except Exception as e:
231         print(f"删除索引时发生未知错误:_{e}", file=sys.stderr)
232
233     sql = f"CREATE_{INDEX}_{index_name}_{ON}_{Passenger}_{passname}"
234     try:
235         execute_sql(conn, sql)
236         print(f"索引_{index_name}_{ON}已创建在_{Passenger}(passname)_{ON}上.")
237         return True
238     except pymysql.MySQLError as e:
239         print(f"创建索引失败:_{e}", file=sys.stderr)
240         return False
241
242
243 def drop_passenger_name_index(conn):
244     """删除 Passenger 表 passname 字段上的索引"""
245     index_name = "idx_passenger_passname"
246     print(f"正在删除索引_{index_name}...")
247     if not conn:
248         return False
249
250     sql = f"DROP_{INDEX}_{index_name}_{ON}_{Passenger}"
251     try:
252         execute_sql(conn, sql)
253         print(f"索引_{index_name}_{ON}已删除.")
254         return True
255     except pymysql.MySQLError as e:
256         if e.args[0] == 1091:
257             print(f"索引_{index_name}_{ON}不存在, 无需删除.")
258             return False
259         else:
260             print(f"删除索引失败:_{e}", file=sys.stderr)
261             return False
262     except Exception as e:
263         print(f"删除索引时发生未知错误:_{e}", file=sys.stderr)
264         return False
265
266
267 # --- 查询测试与结果记录函数 ---
268 def run_query_test(conn, scale, target_name, index_exists):
269     """运行查询测试并记录耗时"""
270     sql = "SELECT_{*}_{FROM}_{Passenger}_{WHERE}_{passname}_{=}_{%s}"

```

```

271
272 query_times_millis = []
273
274 print(f"正在执行{NUMBER_OF_QUERIES_PER_SCALE}次查询(针对'{
    target_name}')...")
275
276 if not conn:
277     return
278
279 try:
280     with conn.cursor() as cursor:
281         for i in range(NUMBER_OF_QUERIES_PER_SCALE): # 执行多次
            查询并记录耗时
282             start_time = time.perf_counter() # 参数化查询, 注意
                参数必须是元组 (即使只有一个元素)
283             # 遍历结果集以确保数据库实际执行了查询并返回了所有匹
                配的行
284             cursor.execute(sql, (target_name,))
285             results = cursor.fetchall() # 获取所有结果
286             end_time = time.perf_counter()
287             query_time_millis = (end_time - start_time) * 1000 #
                记录毫秒
288             query_times_millis.append(query_time_millis)
289             # print(f" 查询 {i+1}/{NUMBER_OF_QUERIES_PER_SCALE}
                耗时: {query_time_millis:.3f} ms, 找到 {len(
                results)} 条记录.") # 打印每次查询耗时和找到的记录
                数
290
291 average_time_millis = sum(
292     query_times_millis) / NUMBER_OF_QUERIES_PER_SCALE
293
294 print(f"平均查询耗时:{average_time_millis:.3f}ms")
295
296 try:
297     with open(RESULT_FILE, 'a', newline='', encoding='utf
        -8') as csvfile:
298         writer = csv.writer(csvfile)
299         # 记录数据规模, 索引是否存在(布尔值), 平均耗时
300         writer.writerow(
301             [scale, index_exists, f"{average_time_millis
                :.3f}"])
302 except Exception as e:

```

```

302         print(f"记录结果到文件失败:_{e}", file=sys.stderr)
303     except pymysql.MySQLError as e:
304         print(f"执行查询测试失败:_{e}", file=sys.stderr)
305
306
307 # --- 主程序逻辑 ---
308
309 if __name__ == "__main__":
310     print("22281052_索引实验开始")
311
312     # 初始化结果文件头部
313     try:
314         with open(RESULT_FILE, 'w', newline='', encoding='utf-8') as
315             csvfile: # 'w' 表示覆盖模式
316             writer = csv.writer(csvfile)
317             writer.writerow(["DataScale", "IndexExists", "
318                             QueryTimeMillis"])
319             print(f"结果文件_{RESULT_FILE}_已初始化.")
320     except Exception as e:
321         print(f"无法初始化结果文件:_{e}", file=sys.stderr)
322         sys.exit(1)
323
324 conn = get_connection()
325 if conn:
326     try:
327         print("数据库连接成功.")
328         # --- 准备基础数据 (如果需要) ---
329         # 您的 Passenger 表本身没有外键。但是Flight 表的数据需要
330         # Airport 和 Airline,
331         # 这里专注于 Passenger 表的实验，因此不强制插入其他表的数据。
332
333         if not BASE_PASSENGER_DATA:
334             print("错误: _BASE_PASSENGER_DATA_样本数据为空，无法进
335                 行实验。", file=sys.stderr)
336             sys.exit(1)
337
338         print(f"已加载_{BASE_DATA_SIZE}_条基础_Passenger_数据样本
339             .")
340
341         target_name_exists_in_sample = any(

```

```

337         record[1] == TARGET_NAME for record in
            BASE_PASSENGER_DATA)
338
339     if not target_name_exists_in_sample:
340         print(
341             f"警告: 查询目标姓名 '{TARGET_NAME}' 不存在于基础
342             样本数据中。请修改 TARGET_NAME。", file=sys.
343             stderr)
344
345     # --- 循环测试不同数据规模 ---
346
347     for scale in DATA_SCALES:
348         if scale < BASE_DATA_SIZE:
349             print(
350                 f"\n--- 数据规模 {scale} 小于基础样本数量 {
351                     BASE_DATA_SIZE}, 跳过 ---")
352             continue
353
354         print(f"\n--- 测试数据规模: {scale} ---")
355
356         if not truncate_passenger_table(conn):
357             print(
358                 f"无法清空 Passenger 表, 跳过数据规模 {scale}
359                 的测试。", file=sys.stderr)
360             continue
361
362         # 插入指定数量的 Passenger 数据, 基于样本
363         if not insert_scaled_passenger_data(conn, scale,
364             BASE_PASSENGER_DATA):
365             print(f"数据插入失败, 跳过数据规模 {scale} 的查询
366                 测试。", file=sys.stderr)
367             continue
368
369         # --- 无索引测试 ---
370         print("\n--- 无索引查询测试 ---")
371         drop_passenger_name_index(conn) # 确保没有索引
372         run_query_test(conn, scale, TARGET_NAME, False)
373
374         # --- 有索引测试 ---
375         print("\n--- 有索引查询测试 ---")
376         create_passenger_name_index(conn) # 创建索引
377         run_query_test(conn, scale, TARGET_NAME, True)

```

```

371         print("\n所有数据规模测试完成.")
372
373     except Exception as e:
374         print(f"实验过程中发生错误:_{e}", file=sys.stderr)
375     try:
376         if conn and not conn.get_autocommit():
377             conn.rollback()
378         if conn:
379             with conn.cursor() as cursor:
380                 cursor.execute("SET_FOREIGN_KEY_CHECKS=1;")
381                 conn.commit()
382     except Exception as cleanup_e:
383         print(f"实验错误后清理失败:_{cleanup_e}", file=sys.
384             stderr)
385     finally:
386         if conn:
387             conn.close()
388             print("数据库连接已关闭.")
389     else:
390         print("无法进行索引实验，数据库连接失败。", file=sys.stderr)
391
392     print(f"\n索引实验结束.结果记录在_{RESULT_FILE}")

```